

AI Meeting Scheduler with Smart Conflict Resolution – Detailed Problem Brief

1. Problem Context

Busy professionals waste significant time scheduling meetings through back-and-forth emails and calendar link exchanges. They must manually describe meeting constraints ("30 minutes, Tuesday or Wednesday after 2 PM, avoid lunch hours"), wait for responses, handle conflicts when attendees aren't available, and then write meeting agendas. Teams often settle for suboptimal times because coordinating multiple calendars manually is exhausting. Currently, tools like Calendly or Google Calendar help find availability but don't understand natural language constraints, propose optimal times intelligently, or generate meeting agendas—users must still do the heavy lifting of context and communication. What's missing is an AI agent that understands meeting requirements described in plain English, intelligently searches for the best time considering constraints and calendars, handles conflicts gracefully when perfect slots don't exist, and auto-generates meeting agendas so the organizer can send a complete, ready-to-go invite in seconds.

2. Objective

Build a web application where users describe a meeting need in natural language (e.g., "I need to meet with the design team for 30 minutes sometime next week, avoid Fridays and mornings"). The AI parses constraints from the request, checks against attendees' calendar availability (mock or connected data), proposes the top 3–5 optimal time slots considering all constraints, generates an AI-written agenda based on the meeting's purpose, and drafts a meeting invite. When perfect slots don't exist, show a conflict-resolution view explaining tradeoffs (e.g., "Slot 1: Works for 4/5 attendees; conflicts with CEO's all-hands").

3. Core Features (Must-Have)

- **Natural Language Meeting Input:** A text input where users describe their meeting need (e.g., "1-hour standup with the backend team Tuesdays at 10 AM, exclude [names]" or "Quick sync with Sarah for 15 min next Tuesday afternoon"). The app accepts freeform, conversational descriptions.
- **Constraint Parsing via LLM:** Uses an LLM to extract structured constraints from the natural language input: duration, attendees, preferred day/time, excluded times/people, other preferences (e.g., "avoid lunch," "before EOD").
- **Mock Calendar Data & Availability Check:** Loads attendees' calendar data (either mocked with sample data or connected to a real calendar service). Identifies available time slots that satisfy all constraints over a specified date range (e.g., "next week," "next month").
- **Optimal Time Slot Proposal:** Generates a ranked list of top 3–5 time slot options, prioritizing by: (1) how many attendees can make it, (2) proximity to stated preferences, (3) fairness (not always early morning, not always late afternoon). Display each slot with a "fit score" (e.g., "95% match").

- **Conflict Resolution View:** When no perfect slot exists where all attendees are free, show a conflict-resolution table: each proposed slot with columns like "Time," "Attendees Available," "Attendees with Conflict," "Severity," and brief explanation. Allow users to see and understand tradeoffs.
- **AI-Generated Agenda:** Based on the meeting's stated purpose (parsed from input), the AI generates a draft agenda with 3–5 bullet points or a structured agenda template (e.g., "Discussion Topics," "Decisions Needed," "Next Steps").
- **Calendar Invite Draft:** Display a preview of the final meeting invite showing: time, attendees, location (if any), meeting title, AI-generated agenda, and meeting notes. Include a "Copy to Clipboard" or "Send Invite" button (for demo purposes, copy-to-clipboard is sufficient).

4. Stretch Features (Good-to-Have)

- **Attendee Calendar Integration:** Connect to Google Calendar or Outlook API (instead of mock data) to fetch real attendee availability. Complex but impressive if time permits.
- **Smart Rescheduling:** If a proposed time doesn't work, allow users to click "Suggest alternatives" and the AI re-ranks slots or asks clarifying questions ("Can we shift to next month?" "Is 6 PM workable?").
- **Meeting Prep Auto-Generation:** Extend the agenda to include pre-meeting preparation tasks or questions to send to attendees 24 hours before, drafted by AI.
- **Conflict Negotiation Suggestions:** When conflicts arise, generate AI-suggested solutions (e.g., "Invite only 3 of 5 attendees and brief others later," "Split into two 30-min sessions").

5. Suggested Tech Stack

Frontend:

- **React** or **Next.js:** Build a clean, interactive UI for describing meetings, viewing proposed slots, and reviewing conflict resolutions. Next.js allows combining frontend and backend.
- **Tailwind CSS:** Quick, professional styling for forms, tables, and conflict-resolution views.
- **React Hook Form:** Simplify the natural language input form.
- **Recharts** or **React Big Calendar** (optional): Visualize proposed time slots as a calendar grid or timeline for better UX.

Backend:

- **Node.js + Express** or **Next.js API Routes:** Lightweight backend for orchestrating LLM calls, constraint parsing, availability checking, and agenda generation.
- Alternatively, **Python + Flask:** Better for data processing and algorithm logic if the team prefers Python.

Database/Storage:

- **Firebase Firestore** or **Supabase:** Store meeting scheduling requests, generated proposals, and user session data. Optional for MVP (could use in-memory storage during hackathon).
- Lightweight: Use browser **localStorage** or **sessionStorage** for MVP.

LLM & AI:

- **OpenAI GPT-3.5 Turbo** or **Claude API**: For constraint extraction, agenda generation, and conflict resolution explanations. Free trial credits available.
- **LangChain** (optional): Simplifies multi-step prompting (e.g., parse input → extract constraints → generate agenda).

Calendar & Scheduling:

- **Mock Calendar Data**: For hackathon MVP, use a hardcoded mock dataset of attendee calendars (JSON objects with time blocks) instead of integrating real APIs. This keeps scope manageable.
- Real Integration (stretch): **Google Calendar API** or **Microsoft Graph API** (Outlook) for connecting to real calendars.

Deployment:

- **Vercel**: Free, seamless Next.js deployment.
- **Firebase Hosting**: Pairs well with Firestore.
- **Heroku** or **Railway**: Simple Node.js/Python deployment.

Why These Choices: Minimal setup friction, generous free tiers, and focus on the core problem (constraint parsing, optimization, LLM use) rather than infrastructure. Mock calendar data lets you build and demo the core logic without API complexity. LangChain simplifies multi-step LLM orchestration.

6. Suggested Approach / Step-by-Step Plan

Phase 1: Setup & Planning (Hours 1–3)

1. **Initialize the project**: Create a Next.js app. Set up GitHub repo, folder structure (pages/components/api/utils/lib), and .env.local for API keys (OpenAI/Claude). Create mock calendar data file (e.g., mockCalendarData.json).
2. **Design the data schema**: Define data structures for:
 - Meeting request: { description, extractedConstraints: { duration, attendees, preferredDays, excludedTimes, ... }, status, createdAt }
 - Proposed time slots: { start, end, attendees, availableCount, totalAttendees, fitScore, conflicts: [] }
 - Mock attendee calendars: { name, busyBlocks: [{ start, end, title }, ...] }
3. **Sketch the LLM prompts**: Write sample prompts offline in ChatGPT/Claude for:
 - Constraint extraction from natural language
 - Agenda generation based on meeting purpose
 - Conflict resolution explanation
 Test these manually to refine before coding.

Phase 2: Frontend & Input (Hours 4–7)

4. **Build the meeting input form page**: Create a React component with:
 - A large text area for natural language meeting description (placeholder text: "e.g., '30 min with design team next Tuesday afternoon, avoid Fridays'")
 - A button "Schedule Meeting"

- Optional: Attendee name input/select (to make it easier than parsing from text)
- Basic form validation (ensure description isn't empty)
- Use Tailwind CSS for clean styling
- Spinner/loading state while processing
- 5. **Build results pages layout:** Create placeholder pages for:
 - **Proposed Slots Page:** Display top 3–5 time slots as cards or a table showing time, attendees available, fit score
 - **Conflict Resolution Page:** Table showing all slots with conflict details (which attendees conflict, severity)
 - **Invite Preview Page:** Show the final meeting invite (time, attendees, agenda, notes)

Phase 3: Backend & Core Logic (Hours 8–16)

- 6. **Implement constraint extraction endpoint** (/api/parse-meeting-request):
 - Accepts natural language meeting description + list of attendee names
 - Calls LLM with a prompt to extract structured constraints
 - Example prompt: "Parse this meeting request and extract: duration (in minutes), attendee names, preferred days/times, excluded times/people, any other constraints. Return JSON: { duration, attendees, preferredDays, preferredTimes, excludedDays, excludedTimes, excludedPeople, additionalNotes }. Request: '[USER INPUT]'"
 - Validates extracted constraints (e.g., duration > 0, attendees list not empty)
 - Returns structured constraints JSON
- 7. **Implement availability checking logic:** Create a function that:
 - Loads mock attendee calendar data (from mockCalendarData.json)
 - For each attendee, identifies their busy blocks for the next 2–4 weeks
 - For a proposed time slot, checks how many attendees are free vs. conflicted
 - Returns availability info: { free: [], conflicted: [], severity: 'low' | 'medium' | 'high' }
- 8. **Implement time slot proposal algorithm** (/api/find-meeting-slots):
 - Accepts extracted constraints (duration, attendees, preferred days, excluded times)
 - Generates candidate time slots (e.g., all 30-min or 1-hour slots over the next 10 business days)
 - For each slot, checks availability using the function from step 7
 - Scores each slot based on:
 - a. % of attendees available (higher is better)
 - b. How close it matches preferred days/times (higher is better)
 - c. Fairness heuristic (avoid always picking early morning; vary)
 - Returns top 5 slots ranked by score with conflict details
 - Example response: { topSlots: [{ time: "Tue Sep 24 2:00 PM", fitScore: 0.95, attendeesAvailable: 4, attendeesConflicted: 1, conflicts: [{ name: "Bob", currentMeeting: "..."}] }, ...] }
- 9. **Implement agenda generation endpoint** (/api/generate-agenda):

- Extracts meeting purpose/title from the original request (e.g., "design review," "standup," "brainstorm")
 - Calls LLM to generate a structured agenda
 - Example prompt: "Generate a meeting agenda for a [MEETING PURPOSE] meeting lasting [DURATION] minutes with attendees [NAMES]. Create 3–5 agenda items with time allocations. Return JSON: { title, items: [{ topic, duration, description }, ...] }"
 - Returns agenda JSON
- 10. Implement meeting invite draft endpoint (/api/draft-invite):**
- Accepts selected time slot, attendees, agenda, meeting purpose
 - Constructs a text-based meeting invite (iCal format is overkill for MVP; plain text/markdown is fine)
 - Returns invite text: subject, time, attendees, location (if any), agenda, notes
 - Example format:
 - MEETING: Design Review Session TIME: Tuesday, September 24, 2:00 PM – 2:30 PM ATTENDEES: Alice, Carol, Dave LOCATION: Conference Room B (or Google Meet link if specified) AGENDA: - Welcome & objectives (5 min) - Design mockup review (15 min) - Feedback & Q&A (10 min) NOTES: Please review the attached mockups before the meeting.

Phase 4: Frontend Wiring & Display (Hours 17–21)

- 11. Wire frontend to backend:** Connect the input form to the backend flow:
- User enters meeting description → Click "Schedule"
 - Frontend calls /api/parse-meeting-request to extract constraints
 - Frontend calls /api/find-meeting-slots with extracted constraints
 - Display proposed slots on results page
 - Allow user to click a slot → calls /api/generate-agenda + /api/draft-invite
 - Display invite preview
- 12. Build conflict resolution UI:**
- Display all proposed slots in a table: Time, Attendees Available (count), Attendees with Conflict (count), Fit Score, "View Details" button
 - When user clicks "View Details" for a slot with conflicts, show a modal/panel listing:
 - Which attendees have conflicts
 - What they're doing at that time (if available in calendar data)
 - Alternative explanation (e.g., "Could reschedule Carol's 2:15 PM standup to 3:00 PM")
- 13. Build invite preview page:**
- Display the final invite as a nicely formatted card/panel
 - Show all details: time, attendees, agenda items, notes
 - Include "Copy Invite to Clipboard" button (for demo; in production could send via email API)
 - Option to edit invite text before copying

Phase 5: Polish & Deployment (Hours 22–24)

14. **Test end-to-end:** Create 2–3 sample meeting requests, verify constraints are parsed correctly, time slots are generated, agendas make sense, and invites are formatted well.

15. **Refinements:**

- Improve error messages (e.g., if parsing fails, prompt user to be more specific)
- Add success messages and confirmations
- Ensure responsive design on mobile
- Clean up UI/styling

16. **Deploy:** Push to Vercel or Firebase. Test the live link.

7. Sample Data / APIs to Use

Mock Calendar Data (for hackathon):

Create a `mockCalendarData.json` file with sample attendee calendars:

```
{
  "attendees": [
    {
      "name": "Alice",
      "busyBlocks": [
        { "day": "Tuesday", "start": "09:00", "end": "10:00",
"\"title\": \"All-hands meeting\" },
        { "day": "Tuesday", "start": "14:00", "end": "15:00",
"\"title\": \"Client call\" },
        { "day": "Wednesday", "start": "13:00", "end": "14:00",
"\"title\": \"Lunch block\" }
      ]
    },
    {
      "name": "Bob",
      "busyBlocks": [
        { "day": "Monday", "start": "10:00", "end": "11:30",
"\"title\": \"Design review\" },
        { "day": "Wednesday", "start": "14:00", "end": "15:00",
"\"title\": \"Standup\" }
      ]
    },
    {
      "name": "Carol",
      "busyBlocks": [
        { "day": "Tuesday", "start": "10:00", "end": "11:00",
"\"title\": \"1:1 with manager\" },
        { "day": "Friday", "start": "09:00", "end": "17:00",
"\"title\": \"Off-site\" }
      ]
    }
  ],
}
```

```

    {
      "name": "Dave",
      "busyBlocks": [] // Dave has a completely open calendar
for demo purposes
    }
  ]
}

```

Sample Meeting Requests (for testing):

- "I need a 30-minute meeting with Alice, Bob, and Carol to discuss the Q4 design roadmap. Sometime next week, but avoid Fridays."
- "Quick 15-min sync with Dave on Tuesday afternoon to review the API spec."
- "1-hour brainstorm with Alice, Bob, and Carol next week in the morning. No Mondays."

LLM Prompts (Sample Templates):

Constraint Extraction Prompt:

You are a meeting scheduling assistant. Parse this natural language meeting request and extract structured constraints.

Meeting Request: "[USER INPUT]"

Extract and return valid JSON (no other text):

```

{
  "duration": number (in minutes),
  "attendees": [string array of names if mentioned, or empty if
not specified],
  "preferredDays": [string array, e.g., ["Monday", "Tuesday"] or
empty for any day],
  "preferredTimes": [string array, e.g., ["morning",
"afternoon"] or "09:00-12:00" format],
  "excludedDays": [string array of days to avoid, e.g.,
["Friday"]],
  "excludedTimes": [string array, e.g., ["lunch", "early
morning"]],
  "excludedPeople": [string array of names to exclude],
  "meetingPurpose": "string (e.g., 'Design review', 'Project
kickoff', 'Sprint planning')",
  "additionalNotes": "string (any other constraints or
preferences)"
}

```

Agenda Generation Prompt:

Generate a meeting agenda for a [MEETING PURPOSE] meeting.

Details:

- Duration: [DURATION] minutes
- Attendees: [ATTENDEE NAMES]
- Key objectives: [EXTRACTED FROM REQUEST if available]

Return valid JSON (no other text):

```
{
  "title": "string (meeting title)",
  "objectives": [string array of 2-3 key objectives],
  "items": [
    { "topic": "string", "duration": number (in minutes),
    "description": "string" },
    ...
  ],
  "preparationNotes": "string (anything attendees should prepare
or review)"
}
```

Make the agenda concise, time-aware (allocations must sum to total duration), and professional.

Conflict Resolution Explanation Prompt:

Explain scheduling conflicts in a way that helps the meeting organizer make a tradeoff decision.

Proposed time slot: [TIME]

Attendees who can make it: [NAMES]

Attendees with conflicts: [NAMES] (conflicts: [DESCRIPTIONS])

Meeting purpose: [PURPOSE]

Suggest one sentence explaining the tradeoff and a potential solution.

Return JSON: { tradeoffExplanation: "string",
suggestedResolution: "string" }

Public Data/APIs:

- **Day.js or date-fns:** For date/time calculations (parsing "next week," "Tuesday," etc.)
- **Open Weather API** (optional stretch): If meeting has location component, fetch weather for outdoor meeting suggestions

Real Calendar Integration (Stretch, if time):

- **Google Calendar API:** <https://developers.google.com/calendar> (OAuth, complex but powerful)
- **Microsoft Graph API (Outlook):** <https://docs.microsoft.com/en-us/graph/>

8. Key Challenges to Anticipate

Challenge 1: Natural Language Parsing Ambiguity

- **Problem:** Requests like "next week" or "morning" are ambiguous. Does "next week" mean Monday-Friday or the literal 7-day period? Does "afternoon" mean 12 PM–5 PM or 1 PM–6 PM? The LLM might interpret these inconsistently across multiple requests.
- **Tip:** Define clear time mappings in your system. Example: "morning" = 09:00–12:00, "afternoon" = 13:00–17:00, "next week" = next 7 calendar days starting tomorrow. Ask the LLM to normalize these to explicit time ranges. Add a clarification step: if parsed constraints are ambiguous, show the user what was extracted (e.g., "I understood: Tuesday or Wednesday, 2 PM–5 PM. Is that correct?") and let them adjust before proposing slots.

Challenge 2: Time Slot Optimization (Multiple Constraints)

- **Problem:** Balancing multiple objectives (maximize attendee availability, respect preferred days/times, avoid always picking early morning, fairness) is non-trivial. Simple greedy algorithms might miss optimal slots. For a hackathon, this could get complicated fast.
- **Tip:** Start with a simple scoring function: $\text{fitScore} = (\text{availableCount} / \text{totalAttendees}) * 0.7 + (\text{preferenceMatchScore}) * 0.3$. Prefer slots where all attendees are free; if none exist, prefer slots where the most people are free. Don't overthink optimization—a simple greedy approach (sort by score, pick top 5) works fine for MVP. If time permits, improve with weighted preference matching (e.g., preferred days get +20 points).

Challenge 3: Conflict Resolution UX Complexity

- **Problem:** Displaying conflict information clearly without overwhelming the user is tricky. Too much detail is confusing; too little and the user can't make decisions.
- **Tip:** Use a tiered disclosure approach: show a simple table (Time, Available/Conflicted counts, Fit Score) first. Add a "View Details" button for each row that expands to show which specific attendees conflict and their current meetings. Color-code rows by severity (green for high availability, yellow for medium, red for low). Keep text concise.

Challenge 4: LLM Output Parsing & Validation

- **Problem:** The LLM might return malformed JSON, missing fields, or inconsistent data (e.g., attendee names that don't match the input, nonsensical time ranges). This breaks downstream logic.
- **Tip:** Always validate LLM responses: check JSON is valid, verify required fields exist, validate values are reasonable (duration > 0, attendee names are in the expected list). If parsing fails, retry once with a more specific prompt or show a user-friendly error: "I

couldn't understand that request. Please be more specific (e.g., 'I need 30 minutes with Alice and Bob on Tuesday morning')." Log failures for debugging.

9. What a Good Demo Looks Like

A compelling demo should show the following flow, all within 3–5 minutes:

1. **Natural Language Input & Parsing (1 min):** The judge enters a conversational meeting request (e.g., "I need a 1-hour design review with Alice, Bob, and Carol. Sometime next week but avoid Fridays and mornings, please"). Show extracted constraints displayed back: "Duration: 60 min, Attendees: Alice, Bob, Carol, Preferred: Mon-Thu afternoon, Excluded: Friday & mornings." This proves the AI understood the request. Alternatively, show a quick "Does this look right?" confirmation screen.
2. **Time Slot Proposals with Clear Tradeoffs (1.5 min):** Display 3–5 proposed time slots as a table or cards showing:
 - Time (e.g., "Tuesday 2:00 PM – 3:00 PM")
 - Fit Score (e.g., "98% match")
 - Attendees Available (e.g., "4/4 available")
 - For slots with conflicts, a quick note (e.g., "Carol has conflict: prep call"). Ideally, show one or two perfect slots (all attendees available) and one slot with a conflict to demonstrate conflict resolution. Click on the conflict slot to expand and show details (which attendee conflicts, their current meeting, severity).
3. **Agenda Generation & Invite Preview (1 min):** Click on a proposed slot. The app generates and displays:
 - A draft meeting agenda with 3–5 bullet points specific to "design review" (e.g., "Q4 mockups review [30 min]", "Feedback & decisions [20 min]", "Next steps [10 min]")
 - A formatted meeting invite ready to copy: subject, time, attendees, location (if any), agenda, prep notes
 - A "Copy Invite" button that copies the invite to clipboard (you can demonstrate by pasting it into a text editor or showing the clipboard contents)
4. **Polish & Efficiency (30 sec):** Show that the entire process—from natural language input to ready-to-send invite—took less than 90 seconds. Emphasize that this would normally take 10–15 minutes of manual email back-and-forth. Responsive design on mobile is a nice plus.

Overall Impression: Judges should think, "This tool actually solves the meeting scheduling problem. The AI understood complex, conversational constraints, intelligently ranked time slots, handled conflicts gracefully, and generated a professional agenda—all in seconds. This is genuinely useful for busy teams." If the constraints are parsed accurately, the time slots are reasonable, the agenda is specific (not generic), and the conflict resolution is clear, you've built something impressive.